

Измерение расстояния с помощью ультразвукового датчика *HC-SR04*, платы *STM32F103* и программы *PicOS-HC-SR04*

Дата: 2026.05.15

Автор: Шендрик Павел Дмитриевич

Оглавление

Введение	1
Схема подключения	2
Описание программы	4
Работа программы	6
Список литературы.....	8

Введение

Для изменения расстояния с помощью платы *STM32F103 BluePill*, также дополнительно необходимы: ультразвуковой датчик расстояния *HC-SR04* [1], программатор *STLINK V2*, преобразователь *USB-TO-UART* и программа *PicOS-HC-SR04*. Для взаимодействия с программой рекомендуется использовать *Terminal v1.9b*.

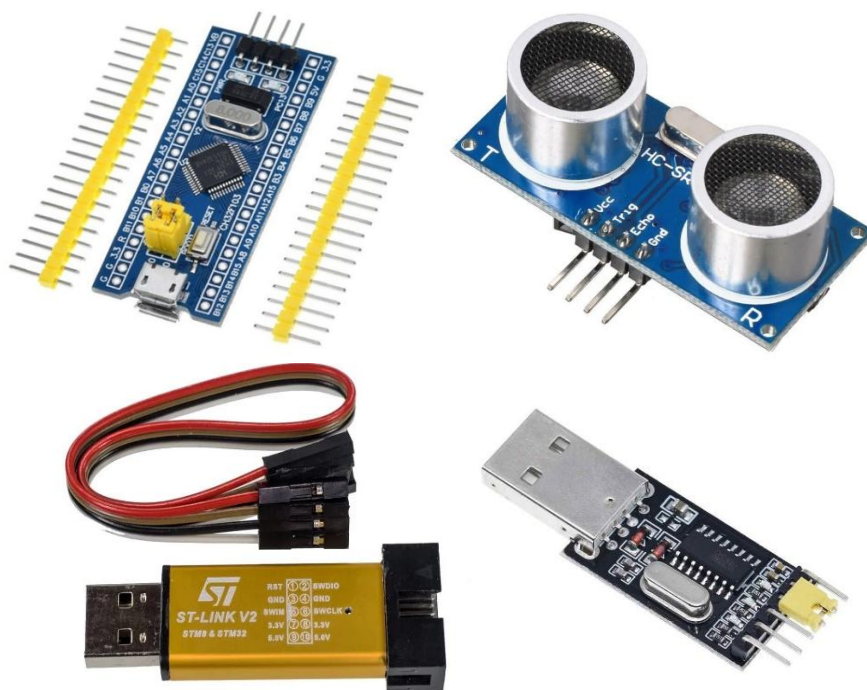


Рис. 1 *STM32F103 BluePill*, *HC-SR04*, *ST-LINK V2* и *CH340* (*USB-TO-UART* преобразователь)

Все датчики расстояния, используемые время полёта сигнала для измерений (ультразвуковые, радиолокационные, некоторые лазерные и редко инфракрасные) измеряют по одинаковому принципу: время пролёта сигнала, зависящее от природы сигнала (скорость звука или света), делённое на два, т.к. сигнал проходит двойное расстояние – до объекта и обратно.

Ультразвуковой датчик расстояния *HC-SR04* для измерения использует два канала первого таймера: *TIM1_CH1* – захватывает передний фронт сигнала ($0 \rightarrow 1$) и записывает значение таймера в регистр захвата *TIM1->CCR1* и *TIM1_CH2* – захватывает задний фронт сигнала ($1 \rightarrow 0$) и записывает значение таймера в регистр захвата *TIM1->CCR2*. Т.к шаг (квант) счёта таймера настроен на 0.5 мкс, то длительность (время пролёта) сигнала в мкс будет вычисляться как: $(TIM1->CCR2 - TIM1->CCR1) * 0.5$, после чего можно будет найти расстояние в мм, разделив длительность сигнала на два и умножив на скорость звука в мм/мкс: $Длительность_в_мкс / 2 * 0.343$. Программа для управления представлена на рисунке 2.

```
// ===== Автомат измерения расстояния до объекта =====
switch (distance_sensor_state) {
    case DISTANCE_SENSOR_START:
        GPIOB->BSRR = GPIO_BSRR_BS10; // Запуск датчика HC-SR04
        delay_us(10 - 1); // Задержка около 10 мкс
        GPIOB->BSRR = GPIO_BSRR_BR10; // Выключение датчика HC-SR04
        TIM1->CNT = 0; // Сброс таймера-счетчика 1
        TIM1->CR1 |= TIM_CR1_CEN; // Включение таймера-счетчика 1
        distance_sensor_state = DISTANCE_SENSOR_WORK; // Изменение состояния
        break;
    case DISTANCE_SENSOR_WORK:
        if (TIM1->SR & TIM_SR_UIF) { // Если произошло событие таймера
            TIM1->SR &= ~TIM_SR_UIF; // Сброс флага события таймера
            if (TIM1->SR & TIM_SR_CC2IF) { // Если получен задний фронт сигнала
                // Длительность сигнала, мкс (0.5 - шаг (квант) счёта)
                distance_sensor_value_mcs = (TIM1->CCR2 - TIM1->CCR1) * 0.5;
                // Измеренное расстояние, мм (длительность / 2 * скорость звука)
                distance_sensor_value_mm = \
                    (uint32_t)distance_sensor_value_mcs / 2 * 0.343;
                TIM1->CR1 &= ~TIM_CR1_CEN; // Выключение таймера-счетчика 1
                distance_sensor_state = DISTANCE_SENSOR_START; // Конец работы
            } else if (TIM1->SR & TIM_SR_CC1IF) break;
            else { // Если произошло переполнение таймера, повторить попытку
                distance_sensor_state = DISTANCE_SENSOR_START; // Перезапуск
            }
        }
        break;
}
```

Рис. 2 Программа для измерения расстояния

Схема подключения

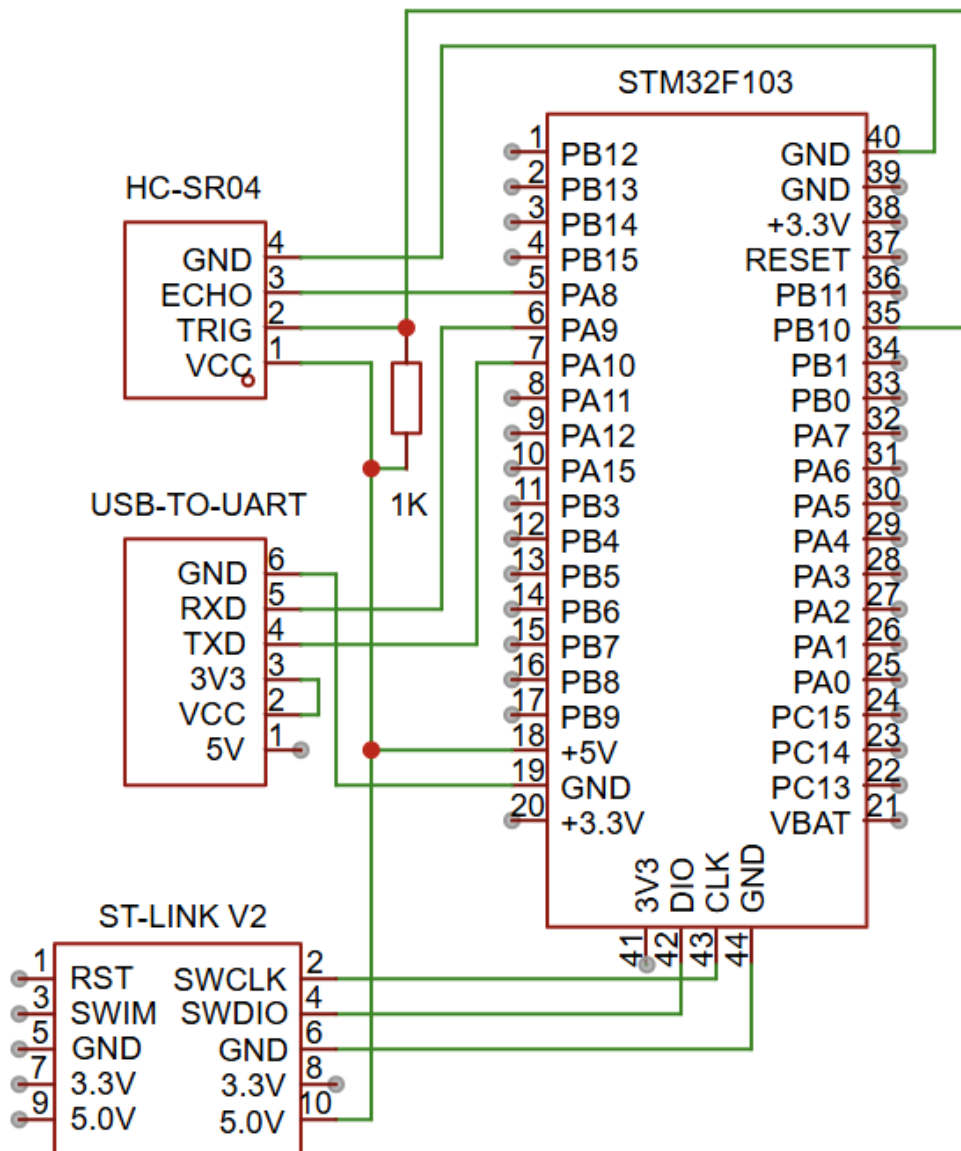


Рис. 3 Схема подключения

Названия портов 41 – 44, могут отличаться от схемы, поэтому необходимо подключать контакты *ST-LINK V2* к соответствующим контактам платы *STM32F103*: *SWCLK* к *SWCLK* (или его сокращению *CLK*, *SWK*, и т.п.); *SWDIO* к *SWDIO* (или его сокращению *DIO*, *D* и т.п.); *GND* к *GND* (или его сокращению *G* и т.п.); «5V» к «5V».

Контакт *TRIG* ультразвукового датчика расстояния *HC-SR04* помимо подключения к контакту платы *PB10*, также подключается к питанию «5V» через резистор номиналом 1 кОм, для повышения напряжения линии с 3.3 В до 5 В.

Описание программы

PicOS – простейшая операционная система (далее «ОС»), обеспечивающая минимальный набор базовых функций. Программа состоит из конечных автоматов [2, 3], постоянно взаимодействующих друг с другом. Главной задачей ОС является общение между системой и пользователем: обработка полученных строк, ответ на команды, вывод и изменение переменных.

В главном цикле *while* (рис. 4) находятся вставки *#include* с содержимым всех автоматов. Такой способ представления информации используется для экономии места, улучшения читаемости главного цикла и ускорения работы системы – во время компиляции программы, вставка заменяется на соответствующий файл, что приравнивается к копированию всего содержимого автомата в *while*, а использование функций, вместо вставок сказывается на быстродействии (т.к. используются вызовы и возвраты из функций), что очень критично для встраиваемых систем. Все автоматы находятся в папке «Tasks», они содержат собственное краткое описание и комментарии к коду.

```
// ===== ГЛАВНЫЙ ЦИКЛ =====  
while (1) {  
    #include "Tasks/usart1_send.c" // Отправка данных по USART1  
    #include "Tasks/usart1_recv.c" // Принятие строки по USART1  
    #include "Tasks/picos.c" // Реализация PicOS  
    #include "Tasks/var_get.c" // Вывод значений переменных по USART1  
    #include "Tasks/var_set.c" // Изменение значений переменных  
    #include "Tasks/led_switch.c" // Изменение состояния светодиода PC13  
    #include "Tasks/distance_sensor.c" // Измерение расстояния до объекта  
}
```

Рис. 4 Главный цикл программы

Все автоматы можно условно разделить на системные, главные и пользовательские:

- Системные автоматы (*usart1_recv*, *usart1_send*, *var_get*, *var_set*) выполняют функции операционной системы: обработку ввода/вывода, управление переменными.
- Главный автомат (*picos*) опрашивает и управляет работой других автоматов, а также анализирует полученные команды.
- Пользовательские автоматы (*led_switch*, *distance_sensor*) реализуют конкретные функции управления или опрашивания по пользовательским командам.

Таким образом, шестой автомат служит для включения и выключения встроенного светодиода *PC13*. Включение светодиода произойдёт после получения команд «led 1» или «led = 1», а выключение – после получения «led 0» или «led = 0». А седьмой нужен для измерения расстояния, с помощью ультразвукового датчика *HC-SR04*. Результат постоянных измерений расстояния можно узнать командой «sensor». Все команды изменяют или запрашивают значения переменных (*led*, *sensor* и т.п.), которые и отслеживаются в программе.

Все переменные, которые можно изменить или запросить командой, описываются в таблице переменных *var_table[]* (рис. 5), каждый элемент таблицы представляет собой структуру из четырёх элементов: Отображаемое имя переменной *name[12]*, краткое описание переменной *description[32]*, тип переменной *type* и указатель на саму переменную **ptr*. Таким образом таблица из переменных *version*, *led* и *sensor* будет выглядеть так:

```
// ===== Инициализация таблицы переменных =====
var_table_t var_table[] = {
    {"version", "Version of the firmware", TYPE_UINT16_T, &fm_version},
    {"led", "LED PC13 state: 1 - ON, 0 - OFF", TYPE_UINT16_T, &led_status},
    {"sensor", "HC-SR04 reading, mm", TYPE_UINT16_T, &distance_sensor_value_mm}
};
```

Рис. 5 Таблица переменных *var_table[]*

Где, *version* – версия программы (равна 32, используется для преобразования в 3.2);

led – статус светодиода: ≥ 1 – включен, 0 – выключен;

sensor – расстояние, измеряемое датчиком *HC-SR04*, в мм;

Таблицу переменных *var_table[]* можно найти в файле «main.c» или вывести командой «help».

Работа программы

После запуска программы автоматы *usart1_send* и *usart1_recv* бездействуют, т.к. их состояния «*usart1_..._state*» равны «*USART1_..._NOT_ACTIVATED*». Первым запущившимся автоматом будет *picos*, который запустит *usart1_send*, для вывода приветственного сообщения, и *usart1_recv*, для получения команд пользователя. По умолчанию состояния автоматов *var_get* и *var_set* также равны «*..._NOT_ACTIVATED*», и они будут неактивны. Их работа регулируется автоматом *picos*, после получения соответствующих команд от пользователя.

Например, после получения автоматом *usart1_recv* команды «*led 1*» и её последующей обработки автоматом *picos*, он запустит автоматы *var_set*, который изменит значение переменной, и *var_get*, который и выведет изменённую переменную с её новым значением. Из-за изменения переменной *led*, сработает автомат *led_switch* и включит встроенный светодиод *PC13*.

На самом деле программа сложнее, т.к. обработка строки происходит за несколько циклов программы, а светодиод включится раньше, чем пользователь получит строку с новым значением переменной.

Для запуска программы:

1. Соберите схему (рис. 6);
2. Вставьте *CH340* и *ST-LINK V2* в компьютер и подключитесь к новому *COM*-порту в приложении *Terminal*;
3. Перепрограммируйте МК запустив *_comp+prog.bat*, после чего в *Terminal* будет выведено приветственное сообщение «Welcome to PicOS v3.2».

Для введения команд рекомендуется использовать окно посимвольного ввода в программе *Terminal* (самое нижнее). Все введённые команды требуют подтверждение – получение системой специального управляющего символа «+CR (Carriage Return)», который можно отправить нажатием на *Enter*, поэтому команду до подтверждения можно исправлять. Корректная работа программы и настройки *Terminal* изображены на рисунке 7.

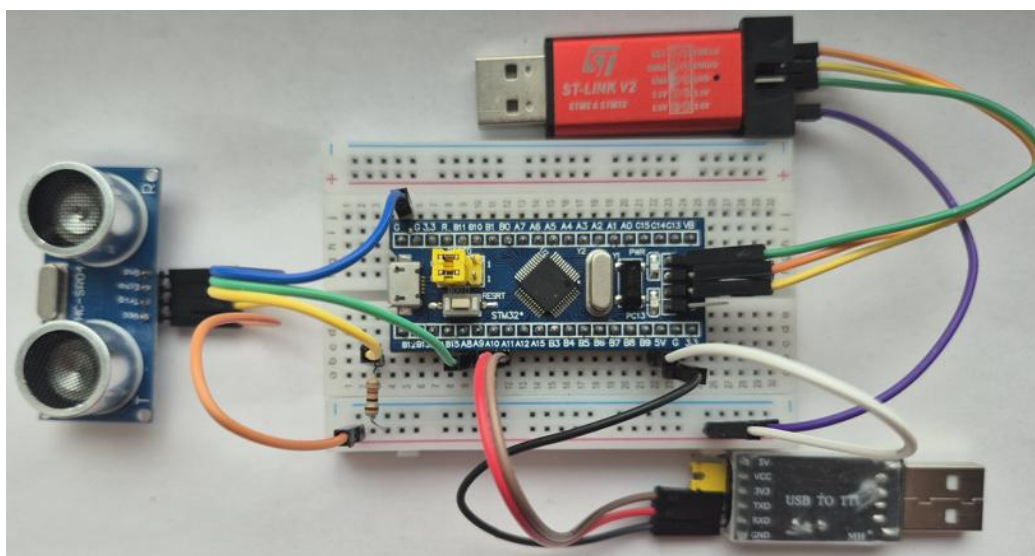


Рис. 6 Собранная схема

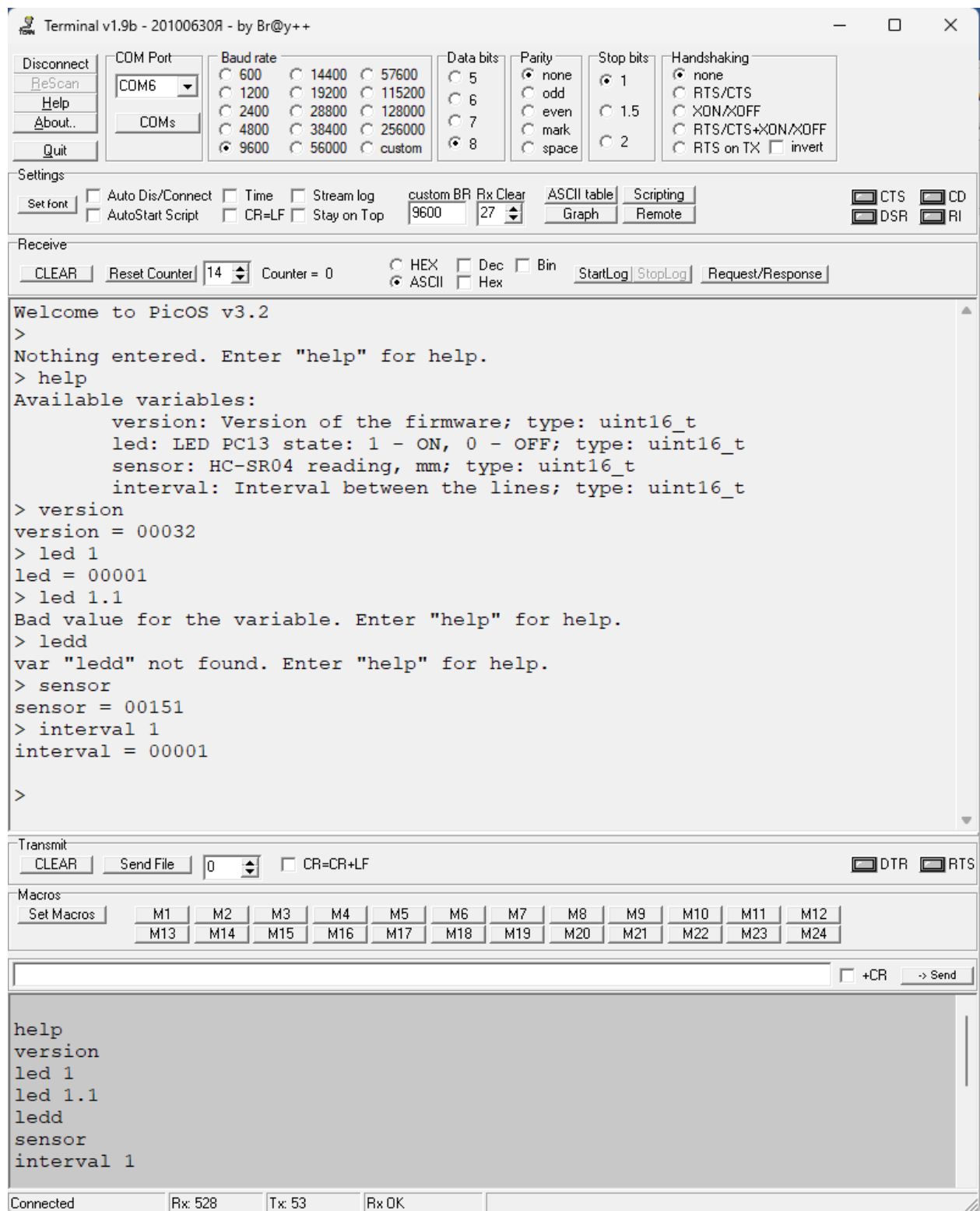


Рис. 7 Результаты работы программы *PicOS-HC-SR04* и настройки *Terminal*

Список литературы

1. Ультразвуковой датчик измерения расстояния HC-SR04 / В.А. Жмудь, Н.О. Кондратьев, К.А. Кузнецов, В.Г. Трубин, Л.В. Димитров // Автоматика и программная инженерия. 2017, №4(22). URL: <https://kb-au.ru/wp-content/uploads/AaSI-2017-4-2.pdf> (дата обращения: 13.10.2025).
2. Введение в автоматное программирование микроконтроллеров STM32 / В.А. Вдовин, И.В. Трубин, В.Г. Трубин, П.Д. Шендрик // Известия Томского политехнического университета. Промышленная кибернетика. –2024. –Т. 2. –No 4. –С. 21–25. DOI:10.18799/29495407/2024/4/75. URL: <https://indcyb.ru/journal/article/view/75/60> (дата обращения: 01.02.2025).
3. Пример псевдопараллельного выполнения трёх задач на микроконтроллере STM32F103 без использования операционной системы / В.А. Вдовин, И.В. Трубин, В.Г. Трубин, П.Д. Шендрик // Известия Томского политехнического университета. Промышленная кибернетика. –2025. –Т. 3. –No 2. –С. 1–7. DOI:10.18799/29495407/2025/2/88 // URL: <https://indcyb.ru/journal/article/view/88/71> (дата обращения: 30.06.2025).